CS 4850 – Spring 2026

SP-14 Blue – Chess AI

Team Members:
Christopher Epps
J'moi Testamark
Ethan McMillian
Paul Porter

Professor:
Sharon Perry

Date:
April 20, 2026

Website:
https://chess-ai-2026.github.io/

GitHub:
https://github.com/Chess-AI-2026/Chess-AI

STATS AND STATUS

Lines of Code: 1200

Components / Tools:
Python
Pygame
Minimax Algorithm
GitHub

Hours Estimated: 257
Hours Actual: 263

Status:
The project is 90% complete and working as designed

Table of Contents

1. Introduction

1.1. Background

Our project is a desktop-based application developed in Python using the Pygame library. Our chess program aims to provide a functional chess environment supporting both local player vs player and player vs AI modes. The AI mechanism utilizes a MiniMax algorithm with Alpha-Beta pruning to generate the different moves. The following document serves as a final comprehensive report covering the development, designs, and validation of the system.

This document defines:

- *The test scope, focus areas, and objectives*
- *The test responsibilities*
- *The test strategy and types of testing used*
- *The entry and exit criteria for testing*
- *The approach used to estimate testing effort*
- *Any risks, assumptions, and dependencies*
- *The testing schedule and major milestones*
- *The test deliverables, including test cases and results*

This document serves as a guide for how testing will be planned, executed, and evaluated for the project.

1.2. Scope

It provides an overview of the chess application.

This document defines:

- *What is tested, including game logic, move validation, AI behavior, and system controls*
- *How testing is performed, including manual functional testing and basic integration testing*
- *What resources are required, including a desktop environment, installation, Python, and Pygame*

The scope focuses on verifying that the chess system functions correctly and meets the project requirements.

2. Implementation Detail Narrative

2.1. System integration and workflow overview:

For our application, we decided to use Python to run our game engine. For our UI, we implemented Pygame for the screen, buttons, and overall aesthetics of our different menus. The main menu includes options for player vs. player, player vs. AI, and quitting. Along with the main menu, the application has menus for choosing AI difficulty, pawn promotion, and deciding when the game is over.

The program begins when the application starts up, and then the engine initializes the board. Upon the user's selection, the engine either opens up with player vs player or player vs AI. After the user moves, the engine determines whether the move is legal or not, and after that, it repeats for the other user or AI.

2.2. Chess Game Engine and Piece Rules:

The engine tracks all the game's legal moves in an array for when a player wants to undo a move. Each type of piece has its own method of how it moves. For pawns, it's complicated since we have to account for promotion and en passant. For the queen, bishop, and rook, since they slide, we implemented a sliding_move method that decides how those pieces move. Since the king is the most important piece, it's a core aspect for checking if the current position is in check, checkmate, or a stalemate.

Some challenges that were faced during the development of the code were making sure PyGame, Python, was installed into Git and the files and folders were named appropriately. And making sure the images were working correctly.

2.3. AI Algorithims:

The core of the AI algorithm is the Minimax algorithm. It operates on the principle of the maximizer, which wants to reach a state with the highest score, while the minimizer wants to reach the lowest score. The AI creates a tree of moves. For each move the AI considers, it simulates every response that the player could make and then every response it would make to those, and so forth. Once the search reaches a left node, it uses the evaluate_board function to turn the board state into a number. And then the number is passed back up the tree. At easy, AI picks a random move; at medium, it only looks two moves ahead; and at hard, it looks three moves ahead. At the hard AI difficulty, we implemented alpha-beta pruning. This optimization ignores branches in the search tree that cannot possibly influence the final decision.

2.3.1. AI logic

1. First, every legal move is identified for the current board state.
2. The AI then places fake moves on a hidden version of the board
3. Calculates the material balance
4. After comparing the scores of all the potential future moves, it selects the best move that leads to the highest guaranteed value.

3. Challenges & Solutions

3.1. Challenges

1. One of the first major problems I had was with a bug where one of the black pawns and black knights could continue moving despite the king being in check.
2. The next issue was loading the main menu and pause menu. The original design loaded the chess game instantly. When including the main menu in the code, the screen would be black. Despite this, the buttons would work and load the chess game. After the game loaded up, the pause menu ended up crashing the program every time the Esc key was pressed.

3. Another major issue that is still being improved upon is the AI being extremely slow at higher depths, which is the exponent for how many future moves the AI is checking. When the AI was first implemented, the hard-difficulty AI would take a minimum of about 30 seconds and upwards of 4 minutes to do one move.

3.2. Solutions

1. The solution we had for the first challenge was to use a method that constantly checked if any of the kings were in check before showing where pieces could move.
2. The solution we had for both the main menu and pause menu was to add another line that manually updated the display when the program started or when a button was pressed. Afterward, we put the entirety of the original code into its own “PLAYING” state and made a “PAUSED” and a “MENU” state that would switch when the corresponding buttons were pressed.
3. This problem is one that we are still currently trying to improve on, but there were some things we did that drastically improved its performance. One being the revision of our first solution. The method to detect any checks was always running, causing some delays. To fix this, we revised the code again to frequently check only when a king was likely to be in check. Another fix was introducing alpha-beta pruning to our minimax method. Lastly, the final fix we have done so far was revising the part of the code that scans and copies the board to make decisions. It now picks up the pieces and checks where it could move rather than making a copy for every possible move.

4. Requirement Summary

4.1. Scope of Recruitment

4.1.1. In scope

The following areas are included in the Requirements for the SP-14 Chess Game AI project:

- *Board initialization and display*
- *Piece movement for all chess pieces*
- *Move validation (legal and illegal moves)*
- *Game rules, including check, checkmate, and stalemate*
- *Special moves such as castling and pawn promotion*
- *Player vs Player gameplay*
- *Player vs AI gameplay*
- *AI move generation and response*
- *Game controls (restart and exit)*
- *Basic system stability (no crashes during gameplay)*

Testing will include functional testing, unit testing of core components, and integration testing of system flow.

4.1.2. Out of scope

The following areas are not included in testing:

- *Multiplayer over a network*
- *User authentication or login systems*
- *Saving and loading games*
- *Mobile platform testing*

Testing is limited to a local desktop environment and focuses on core gameplay functionality.

5. Analysis of Scope

5.1. Release Content

The current release of the SP-14 Chess Game AI project includes the core gameplay features.

Release content includes:

- *Graphical chessboard display*
- *Piece movement for all standard chess pieces*
- *Move validation (legal and illegal moves)*
- *Game rule enforcement (check, checkmate, stalemate)*
- *Special moves (castling and pawn promotion)*
- *Player vs Player mode*
- *Player vs AI mode*
- *Basic AI opponent using a minimax approach*
- *Game controls (restart and exit)*

This release represents a working version with core functionality implemented.

5.2. Regression Testing

Regression testing was performed to ensure that existing features continue to work after new changes are added.

The following areas were tested:

- *Piece movement after rule updates*
- *Move validation after adding new features*
- *Game state detection (check, checkmate) after logic changes*
- *AI behavior after updates to decision logic*

5.3. Platform Testing

Testing performed on a desktop environment.

Platform details:

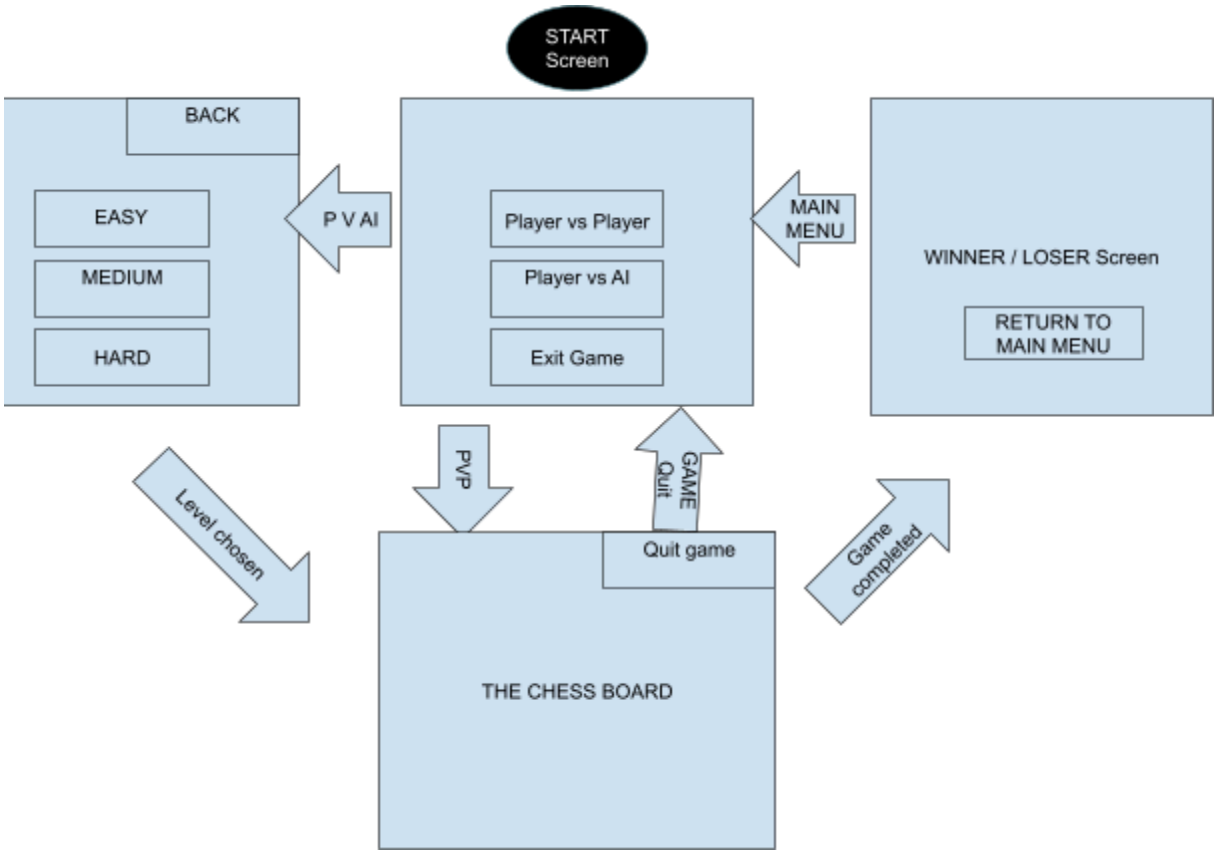
- *Operating System: Windows and macOSv*
- *Hardware: Standard laptop or desktop computer*
- *Programming Language: Python 3*
- *Libraries: Pygame*
- *Input Devices: Mouse and keyboard*

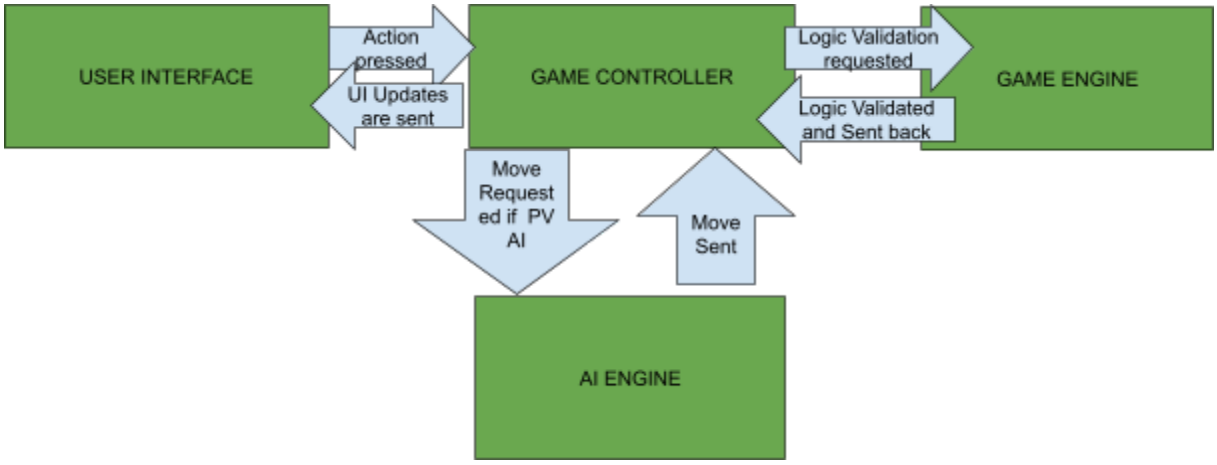
The system is tested locally and does not require internet connectivity.

6. Design

6.1. System Architecture

The system is divided into different parts, including the UI, game controller, game engine, and AI engine. The UI handles rendering and user input. The game controller manages turn order and coordinates between the UI, game engine, and AI engine. The game engine makes sure the chess rules are being maintained. The AI engine evaluates board positions and selects moves. Each subsystem can interact through different method calls. The UI sends user actions to the game controller. The game controller queries the game engine to determine if moves are legal. The game controller requests a move from the AI engine.





6.2. Detailed System Design

6.2.1. Classification

A desktop application utilizes subsystems and modules.

6.2.2. Definition

The chess AI game manages chess gameplay and AI decision-making, allowing player-vs-player or player-vs-AI gameplay.

6.2.3. Constraints

- One active game can be played at a time
- The board cannot be changed (Fixed size and Color)
- AI depth is limited

6.2.4. Resources

Hardware Resources

- The Processor—The system uses the CPU to perform moves, calculations, and game logic. Processing increases with higher search depth.
- RAM—the memory is used to store the current board state, search trees, move history, and evaluation scores.
- Storage—used for program files and for configuration files.

Software Resources

- Operating system—compatible with Windows, macOS, and Linux.
- Programming Environment: Python 3.x interpreter is required

Data Resources

- Game state data—Stores the current board configuration, including piece positions, castling, and move counters. Along with representing using object arrays.
- Move History—Stored in memory during runtime
- Configuration Data—Includes difficulty level, time limits, and UI settings.

6.2.5. Interface/Exports

The chess AI component provides a well-defined set of interfaces that enable interaction between the game engine, user interface, and AI modules. The board management interface allows the components to initialize, retrieve, update, and reset the game board. The game Control interface controls the overall game flow by starting new games, processing player input, switching turns, and determining the end game conditions, such as a win or a draw. The save and load interface, which enables the storage and retrieval of game states through file-based operations. The user interface handles board communication, runtime, and logical errors through structured and exception messages. All interfaces use clearly defined input and output formats, enforce access control to shared data, and ensure consistent communication between system components.

7. Project Setup and Environment Configuration

Prerequisites:

- Python 3.10+
- Libraries: Pygames, Random, Enum
- Hardware: Minimum 8 GB RAM

Installation steps:

- Clone the Repository
  - Download the project from the GitHub organization:
- Run Application
  - Execute through the main class, and the application will start.
- Launch Game
  - The application window opens and displays the chessboard.
- Select Game Mode
  - The user chooses either player vs player or player vs AI.

8. Progression Test Objectives

This section details the progression test objectives.

Ref	Function	Test Objective	Evaluation Criteria	X-Ref	P
Function to be tested					
PT-1	Board Initialization	Verify board loads correctly	The board displays an 8x8 grid with the correct starting positions	SRS 3.1	High

PT-2	Piece Movement	Verify pieces move according to the rules	Legal moves are accepted and reflected on the board	SRS 3.4	High
PT-3	Illegal Move Handling	Verify illegal moves are rejected	Invalid moves are blocked, and no state change occurs	SRS 3.4	High
PT-4	Check Detection	Verify that the system detects the check	The system correctly identifies when the king is in check	SRS 3.5	High
PT-5	Checkmate Detection	Verify the system detects checkmate	The game ends when the checkmate condition is met	SRS 3.5	High
PT-6	Special Moves	Verify castling and pawn promotion	Special moves execute correctly when conditions are met	SRS 3.5	Medium
PT-7	AI Move Generation	Verify AI selects valid moves	AI produces legal moves within a reasonable time	SRS 3.6	High
PT-8	Game Restart	Verify game reset functionality	The board returns to the initial state	SRS 3.7	Medium
PT-9	Game Exit	Verify application exit	The application closes without error	SRS 3.7	Low

9. Regression Test Objectives

This section details the regression test objectives.

Ref	Function	Test Objective	Evaluation Criteria	X-Ref	P
Regression testing					
RT-1	Piece Movement	Ensure movement still works after updates	Previously valid moves still work correctly	SRS 3.4	High

RT-2	Move Validation	Ensure illegal moves are still rejected	Invalid moves are still blocked	SRS 3.4	High
RT-3	Game Rules	Ensure rule logic remains correct	"Check" and "checkmate" are are still detected correctly	SRS 3.5	High
RT-4	AI Behavior	Ensure AI still produces valid moves	AI returns legal moves after updates	SRS 3.6	High
RT-5	Game Restart	Ensure restart still resets the game	The board returns to the initial state	SRS 3.7	Medium
RT-6	UI Updates	Ensure UI still updates after moves	The board refreshes correctly after each move	SDD Section 4	Medium

10. Other Testing

10.1. Security

Basic security testing ensures the application does not crash or behave unexpectedly with invalid inputs.

10.2. Stress & Volume Testing (S&V)

Stress testing for extended gameplay sessions.

Testing included:

- *Running the game for long periods*
- *Performing many moves in sequence*
- *Repeatedly restarting the game*

Outcome:

- *The application remains stable*
- *No crashes or major slowdowns occur*

Testing performed by the developers.

10.3. Unit Testing

Unit testing was performed on individual components of the system.

Components tested include:

- *Piece movement logic*
- *Move validation logic*
- *Game state detection (check, checkmate)*
- *AI decision logic*

Each unit was tested independently to ensure correct behavior.

10.4. Integration Testing

Integration testing verifies that system components work together correctly.

Testing included:

- *UI interaction with the Game Controller*
- *Game Controller interaction with Game Engine*
- *AI Engine integration with game state*
- *Full gameplay flow from move input to board update*

Outcome:

- *All components communicate correctly*
- *The system functions as a complete application*

11. Test Strategy

11.1. Test Level Responsibility

Primary (P) and secondary (S) responsibility for performing testing:

Test Level	External Party	Proj Team	Business
Unit Testing		P	
Integration Testing		P	
Security Testing		P	
Connectivity Testing		P	
User Acceptance Testing		P	
Production Verification Testing		P	

All testing is performed by the project team.

11.2. Test Type & Approach

The types of testing covered by the project team and the standard objectives:

Test Type	Objectives
Functional Testing	Verify that the system meets requirements, performs correctly, handles errors, and enforces game rules
Regression Testing	Ensure existing features continue to work after changes
Unit Testing	Verify individual components such as move logic and AI behavior
Integration Testing	Ensure components work together correctly

11.3. Build Strategy

The system was developed as a single desktop application.

Features implemented in stages:

- *Core board and movement*
- *Rule validation*
- *AI functionality*
- *Testing and refinement*

Each stage was tested before moving to the next.

11.4. Test Execution Schedule

Testing occurred throughout development:

- *Early Stage: Unit testing of core components*
- *Mid Stage: Integration testing of system flow*
- *Late Stage: Full system testing and regression testing*

11.5. Facility, data, and resource provision plan

11.5.1. Test environment

- *Desktop computer (Windows or macOS)*
- *Python 3 installed*
- *Pygame library installed*

11.6. Testing Tools

The following tools were used for testing:

Process	Tool

Test case creation	Microsoft Word
Test tracking	Microsoft Excel
Test execution	Manual
Defect tracking	Microsoft Excel
Version control	Github

12. Software Test Report

12.1. Test Environment Details

12.1.1. Testers

Number of testers: 2 (developers)

System access: Local access to the application and source code

Hardware: Laptop or desktop computer

12.1.2. Hardware and Firmware

Device: Personal laptop or desktop

Purpose: Run and test the chess application

Availability: Provided by the developer throughout the project duration

No special firmware or equipment required

12.1.3. Software

- *Operating System: Windows or macOS*
- *Programming Language: Python 3.13*
- *Library: Pygame*
- *Tools: GitHub, Microsoft Word, Microsoft Excel*

Software is installed locally by the developers before testing begins.

12.1.4. Other Materials

- *Chess rules reference (for validation)*
- *Development documentation (SRS, SDD)*

12.2. Establishing Environment

The plan for establishing the testing environment and responsibilities.

Task	Requirements	Responsibility
Install Python	Python 3 Installed	*Developers*
Install Pygame	Library Installed	*Developers*
Setup Project Files	Source code installed	*Developers*
Run application	Verify Program Runs	*Developers*

12.3. Environment Control

- *Code is managed using GitHub*
- *Only stable versions are tested*
- *Testing is performed on a local controlled environment*
- *Changes are tracked through version control*

12.4. Environment Roles and Responsibilities

The roles and responsibilities of persons who will be responsible for, or interface with, the environment

Role	Staff Member	Responsibilities
Test Manager	Ethan McMillian/Paul Porter	Responsible for executing testing and tracking results
QA Analyst	J'moi Testamark /Christopher Epps	Responsible for creating test cases
Project Manager	Christopher Epps	Escalation point for environment issues.

12.5. Test Cases' Results

This section records the results of executed test cases. Each test case is evaluated based on whether it passed or failed and the severity of any issues found.

Test Case	Description	Pass	Fail	Severity

TC-1	Verify that the application initializes correctly and displays a standard 8x8 chess board with all pieces in their correct starting positions	Yes		None
TC-2	Verify that valid piece movements (based on chess rules) are accepted and correctly update the board state	Yes		None
TC-3	Verify that invalid or illegal moves are rejected and do not alter the game state	Yes		None
TC-4	Verify that the system correctly detects when a king is in check and notifies the player	Yes		None
TC-5	Verify that the system correctly detects a checkmate condition and ends the game appropriately	Yes		None
TC-6	Verify that the AI opponent generates valid moves based on the current board state and executes them without errors	Yes		None
TC-7	Verify that the restart function resets the board to its initial state and clears all previous moves	Yes		None
TC-8	Verify that the application exits cleanly without errors or crashes when the exit command is triggered	Yes		None

13. Assumptions and Dependencies

13.1. Assumptions

- The application will run on a desktop environment
- Python and Pygame are installed correctly
- The tester understands basic chess rules
- The system is tested locally with no network dependency
- The developer is available to fix issues during testing

13.2. Dependencies

- Python 3 must be installed
- Pygame library must be installed
- Source code must be accessible
- The development environment must be properly configured

14. Entry and Exit Criteria

14.1. Entry Criteria

- Application builds successfully
- Core features are implemented
- The test environment is set up
- Required tools are installed

14.2. Exit Criteria

- All test cases are executed
- Major defects are resolved
- No critical system failures remain
- System performs core functionality correctly

15. Definitions

The following acronyms and terms have been used throughout this document:

Term/Acronym	Definition
AI	Artificial Intelligence
UI	User Interface
PvP	Player vs Player
PvAI	Player vs AI
STP	Software Test Plan
STR	Software Test Report

16. Version Control

GitHub was used for version control. The repository was used to store, manage, and track all code throughout the development process. The team utilized commits to record incremental changes. Major components such as game logic, move validation, and AI functionality were developed and updated in stages, with each update committed to the repository. Version control also provides a way to safely test and refine features without losing previous working versions of the code. In the event of issues or bugs, earlier versions could be restored. The repository link is provided on the cover page.

17. Conclusion

The chess AI application has successfully met all the objectives we set during our development phase. Our program is reliable and abides by the chess rules and regulations. The final application fully supports the full game rules, including castling and pawn promotions. The full game engine is fully integrated with a graphical user interface and an artificial intelligence opponent.

The project is currently 90% complete and functions as intended. The successful completion of this AI is the culmination of technical skills in computer science. Future enhancements could include online multiplayer and a way to save and load the current game board.

18. References

The following documents have been used to assist in the creation of this document.

#	Document name	Version	Comments
1	SRS Document	1	Requirements
2	SDD Document	1	System Design
3	Chess Rules	N/A	Game rules reference